ChatGPT

Analytical Report on Recreating Local Workflow Automation Apps

Executive Summary

You asked for a rigorous plan to recreate the **base functionality** of the apps described in the attached files and run the clone **locally on your own computers**. The uploaded materials point not to a simple single-purpose app, but to an **n8n-style workflow orchestration platform**: a visual canvas for triggers and actions, branching logic, data mapping, custom code steps, webhook/schedule/manual triggers, durable execution, credential storage, execution history, real-time run visualization, and optional AI orchestration. The architecture notes in the files also emphasize a split between UI/API, queueing, workers, and durable state. filecite turn0file0 filecite turn0file1

The best engineering path for a **local-first, reproducible, maintainable clone** is a **modular monolith** with intentional process boundaries from day one: a React-based visual editor, a TypeScript backend, PostgreSQL for durable relational state, Redis with BullMQ for queued execution, local file storage or MinIO for blobs, encrypted credential storage, and SSE/WebSocket execution streaming. Package it first as a **local web app launched through Docker Compose**; add **Tauri** packaging later if you need a native desktop shell. This gives you the lowest complexity-to-capability ratio while preserving a clean path toward multi-user or clustered deployment. The attached blueprint strongly supports this split architecture, including React/React Flow on the frontend, Redis/BullMQ-backed workers, and PostgreSQL-backed execution state. filecite turn0file1

For LLMs, the most practical approach is **Ollama-first with provider adapters**. Keep local models as the default for privacy-sensitive, offline, and low-cost workflows, then add optional cloud connectors for **OpenAI** and **Azure AI Foundry** behind the same provider abstraction. Ollama exposes a local runtime and REST API for running local models; OpenAI's GPT-4.1 family introduced very long-context options, and GPT-4o mini was announced at low token cost; Azure AI Foundry has expanded into a multi-model catalog that includes third-party and frontier models, which is strategically useful if you want enterprise controls or a Microsoft-centric environment. [1]

A respectful pushback: if the actual business goal is **"a private, locally hosted automation platform we can control"**, a full greenfield clone is usually **not** the fastest or cheapest route. The attached research shows that **Activepieces** and **Windmill** are materially more open for extension than Zapier/Make and less licensing-constrained than n8n's fair-code model, while still exposing much of the architecture you want. If speed matters more than originality, **forking/extending Activepieces or building on Windmill** is often the higher-return decision. filecite turn0file0 filecite turn0file1

Critique of the recommended path

This recommendation is deliberately conservative. It optimizes for **local reliability, debuggability, and time-to-first-working-system**, not for the flashiest architecture. Its main weakness is that it will feel "less

pure” than a fully decomposed microservice design and less “native” than starting with Electron or Tauri on day one. But the attached materials repeatedly show that the main difficulty in these products is not window chrome or service decomposition; it is **durable execution, queued orchestration, secrets handling, code sandboxing, and visual workflow correctness**. In other words, the recommendation biases toward solving the hard parts first. filecite turn0file1

Reconstruction Scope and Minimal Viable Product

The uploaded files collectively describe a product category centered on **visual workflow automation/ orchestration** across API integrations, branching, code execution, and AI-enhanced flows. The comparison document highlights n8n, Zapier, Make, Activepieces, Pipedream, and Tray.ai as the relevant comparison set, while the architectural blueprint goes deeper into workflow entities, queue mode, workers, sandboxes, waitpoints, and AI orchestration patterns. filecite turn0file0 filecite turn0file1

The minimum viable clone should therefore reproduce the **core functional loop** of these tools rather than their entire ecosystem. In practical terms, that loop is: **design a workflow visually, configure inputs and credentials, run it manually or from a trigger, process data through a graph, observe results in real time, and resume or retry when failures occur**. The attached files explicitly call out visual builders, branching, data mapping, custom code, queue management, state tracking, credential management, webhook handling, and execution visualization as foundational. filecite turn0file0 filecite turn0file1

Capability	Why it is core	MVP decision
Visual canvas with nodes and edges	This is the defining UX of the product category	**Build now**
Manual run, webhook trigger, and schedule trigger	Gives immediate utility and proves the execution model	**Build now**
Branching and merge nodes	Required to reproduce real automation logic	**Build now**
Data mapping and expression evaluation	Without it, workflows are not reusable or composable	**Build now**
Code step	Necessary for parity with n8n/Activepieces-style flexibility	**Build now**, but sandboxed
Credential store	Required for connectors and any serious automation	**Build now**
Execution history and logs	Required for debugging and trust	**Build now**
Real-time node-by-node execution status	Strongly expected by users of this category	**Build now**
Waitpoints and resumable runs	Important, but can be limited in the first release to delay + webhook resume	**Build partial now**

Capability	Why it is core	MVP decision
Large integration marketplace	High-value, but expensive to build and maintain	**Defer**
Multi-user RBAC, SSO, audit trails	Important for team use, but not for local-first single-user parity	**Defer unless needed**
AI workflow builder and multi-agent planner	Valuable differentiator, not required for base cloning	**Defer to phase two**

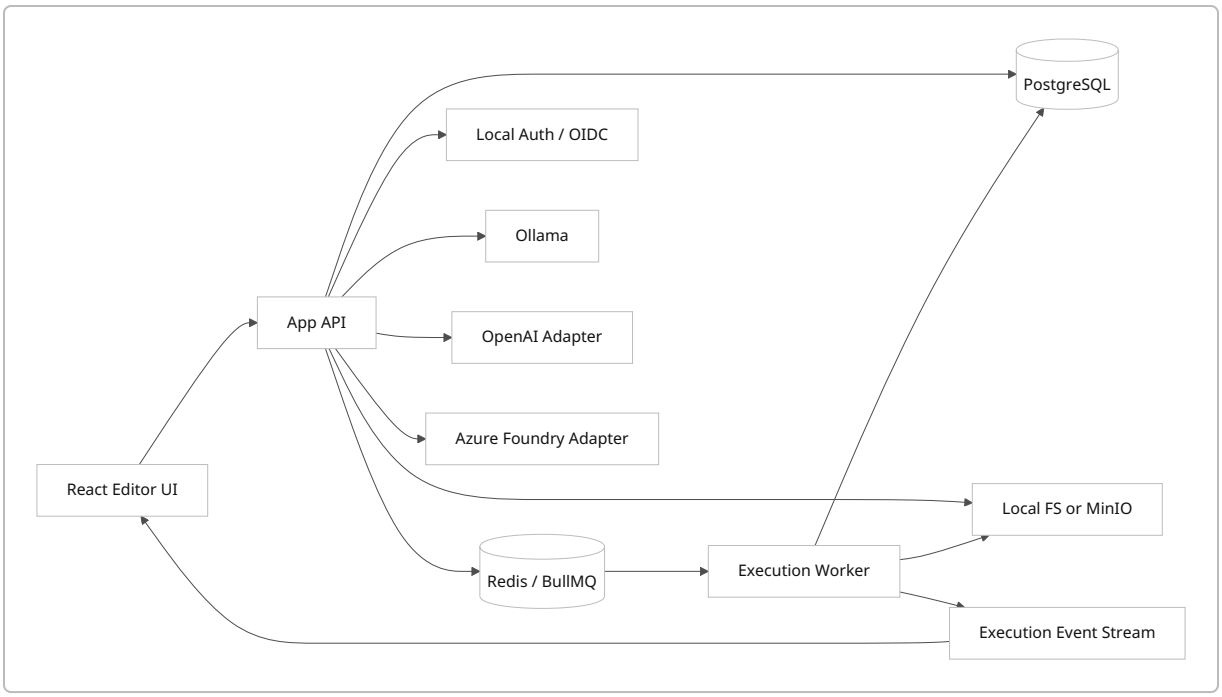
This table is grounded in the uploaded comparison and architecture material, which explicitly identifies the visual builder, branching, data mapping, custom code, webhook handling, queueing, secrets, and execution visualization as the real “must-have” substrate for an n8n-class clone. filecite turn0file0 filecite turn0file1

A realistic MVP node set should stay intentionally small. Start with roughly **fifteen to twenty nodes**: Manual Trigger, Webhook Trigger, Cron Trigger, HTTP Request, If, Switch, Merge, Delay, Set/Transform JSON, Loop over Items, Code, Log/Debug, SQLite or Data Table access, File Read/Write, LLM Chat, and one or two connector exemplars such as Gmail or Slack equivalents if you need external-system demos. That is enough to prove the platform’s base functionality without creating a connector-maintenance trap on day one. The attached files repeatedly show that platform quality comes from the **engine and editor**, not the absolute count of nodes in the first version. filecite turn0file0 filecite turn0file1

Architecture and Recommended Stack

The cleanest local architecture is a **modular monolith with split workers**. That means one codebase, one deployment definition, and one primary database schema, but at least two runtime roles: **app/API/editor** and **worker/executor**. This is the same architectural direction shown in the uploaded blueprint: a frontend/API tier, Redis queue tier, worker fleet, and PostgreSQL-backed state. The monorepo guidance in the file also maps naturally to this approach, with separate packages for web, server, worker, engine, sandbox, shared schemas, and optional enterprise features. filecite turn0file1

A pure monolith is simpler to start, but if it runs webhooks, cron, editor UI, and code execution in one process, it quickly becomes fragile under load. A full microservices design, on the other hand, is unnecessary for local-first development and raises operational overhead too early. The middle ground is best: **one repository, one logical application, multiple runtime roles**, and explicit interfaces for queueing, execution, pub/sub, and storage. That preserves local simplicity while following the queue-backed patterns described in the files. filecite turn0file1



The stack choice should optimize for **editor ergonomics, execution determinism, and local operability**. A TypeScript-heavy stack makes this unusually attractive because it lets one language span editor schemas, workflow JSON, runtime nodes, validation, and worker execution. The attached monorepo and canvas recommendations align strongly with this. filecite turn0file1

Stack option	Frontend	Backend	Queue	DB	Best use	Trade-off
Recommended	React + React Flow + Vite or Next.js	Fastify or NestJS in TypeScript	Redis + BullMQ	PostgreSQL	Fastest route to n8n-like parity	JS sandboxing needs extra care
Python-heavy	React + React Flow	FastAPI + Celery/RQ	Redis	PostgreSQL	Teams stronger in Python/ AI	More friction for shared workflow schemas
Go-heavy	React + React Flow	Go API + worker engine	Redis/ NATS queue	PostgreSQL	Lower runtime overhead	Slower feature iteration in editor-heavy product

The uploaded blueprint explicitly cites React/React Flow for the canvas, Redis/BullMQ workers, PostgreSQL for durable state, and a modular monorepo for separation of concerns. filecite turn0file1

My concrete stack recommendation is this:

Layer	Recommendation	Why
Frontend	React, TypeScript, React Flow, Zustand or Redux Toolkit, TanStack Query	Strong fit for node-canvas UIs and state-heavy editors
Backend API	Fastify or NestJS	Good schema-centric APIs, TypeScript reuse, strong local dev ergonomics
Workflow engine	TypeScript service package	Shared node schemas, shared validation, one language across editor and engine
Queue	Redis + BullMQ	Matches the architecture documented in the uploaded files
Database	PostgreSQL locally in Docker; SQLite only for very small single-user desktop mode	PostgreSQL fits execution history, credentials, workflow versioning, logs
Blob storage	Local filesystem first; MinIO optional later	Lowest local complexity
Auth	Single-user bootstrap auth first; optional OIDC later via Authentik/Keycloak	Best fit for local-first rollout
Real-time	SSE first, WebSockets later if you add collaboration/chat	SSE is simpler for execution streaming
Desktop shell	Tauri later	Smaller footprint than Electron for this use case

The choice of **PostgreSQL over SQLite** for the main recommended path deserves emphasis. SQLite is excellent for a truly single-user desktop app, but the attached blueprint's data model includes projects, workflows, versions, credentials, executions, execution payloads, and webhooks. That model naturally maps better to PostgreSQL once you introduce workers, live execution streams, and resumable runs. filecite turn0file1

Local Deployment Approaches

You asked explicitly about native desktop shells, local web apps, containerized services, and light orchestration. The right answer is not one of these in isolation, but a **staged deployment ladder**: begin with a **local web app on Docker Compose**, then optionally wrap it with **Tauri**, and reserve **k3s** for teams that genuinely need multi-node parity or Kubernetes workflow practice. Docker Compose remains the fastest way to define and launch multi-container local services from YAML, while Podman offers a daemonless, rootless alternative with strong Docker compatibility. Tauri uses local webviews with a Rust backend and is generally leaner than Electron, which ships Chromium and Node.js together. k3s has shown low resource consumption among lightweight Kubernetes distributions, but it still adds material operational overhead. [2]

Approach	What it looks like locally	Strengths	Weaknesses	Verdict
Local web app	Browser UI on `localhost`, backend + DB local	Fastest to build, easiest to debug, easiest to script	Feels less “productized” than desktop shell	**Best starting point**
Tauri desktop app	Bundled native shell around local web UI/API	Smaller footprint, native installer, stronger OS integration	Adds packaging/ signing complexity	**Best secondary packaging choice**
Electron desktop app	Chromium + Node.js desktop shell	Mature ecosystem, many desktop integrations	Heavier memory/ storage footprint	Use only if Tauri blocks you
Docker Compose services	App, worker, Redis, Postgres, Ollama in containers	Repeatable, team-friendly, easiest infra parity	Requires container runtime	**Default local deployment**
Podman / Podman Desktop	Docker-like local container workflow, rootless-friendly	Daemonless, rootless security posture	Slightly less ubiquitous than Docker in tutorials	Strong alternative to Docker
k3s	Lightweight k8s cluster on one or more machines	Best for orchestration learning and cluster parity	Overkill for MVP and most local teams	Use only after Compose works

Source basis for this comparison: Docker Compose’s YAML-based multi-container model, Podman’s daemonless and rootless design, Tauri’s WebView-plus-Rust architecture, Electron’s Chromium-plus-Node architecture, and recent benchmark work showing k3s’s low resource profile among lightweight Kubernetes distributions. [2]

For distribution inside your team, I would adopt this rule of thumb:

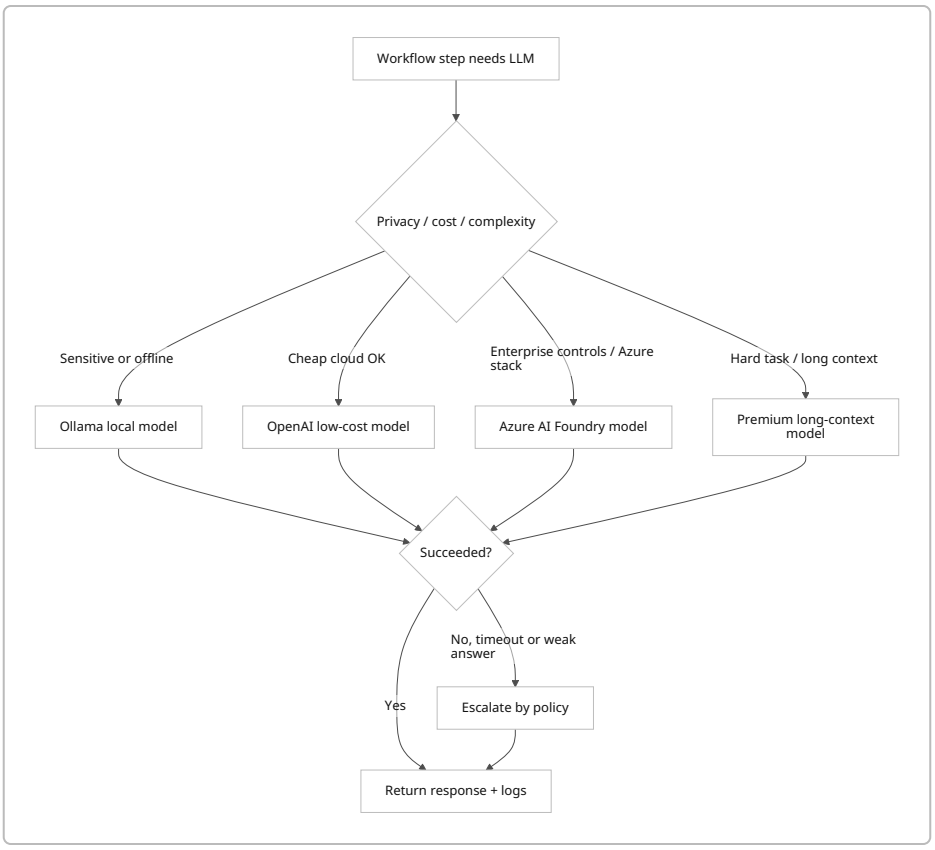
- **Developers** run the stack through Docker Compose or Podman Compose.
- **Non-developers** get a Tauri-packaged desktop app that starts or checks the local services.
- **Ops-minded teams** may maintain a k3s profile only after the Compose profile is stable.

That approach minimizes operational branching without dragging everyone into Kubernetes too early. [3]

LLM Integration Strategy

A local clone should treat LLM access as a **provider-agnostic service layer**, not as hardcoded node logic. The execution engine should call an internal **model gateway** with a normalized schema for chat, completions, embeddings, tool calls, structured output, and streaming. That gateway then routes to **Ollama**, **OpenAI**, or **Azure AI Foundry** based on policy, availability, privacy sensitivity, and budget. Ollama is well suited to local-first runtime because it exposes a local REST API and model-management workflow

for running open-weight models on macOS, Linux, and Windows. OpenAI's GPT-4.1 family introduced a one-million-token context window in public announcements, and GPT-4o mini was announced with low per-token pricing. Azure AI Foundry has increasingly become a multi-model broker, hosting a growing set of models beyond OpenAI alone. [4]



A practical model matrix for this system is below.

Channel	Representative models	Context window	Cost posture	Latency posture	Good default use
Ollama local	Llama 3.1 8B, Gemma 3 12B, Qwen2.5 7B/14B long-context variants	128K for Llama 3.1; 131,072 for Gemma 3; up to 1M in Qwen2.5-1M family	No per-token provider fee; hardware/ power cost only	Highly hardware-dependent	Private drafting, routing, extraction, local coding help

Channel	Representative models	Context window	Cost posture	Latency posture	Good default use
OpenAI	GPT-4o mini	128K-class public launch generation; verify current catalog before rollout	Low API cost; Reuters reported $0.15 input / $0.60 output per 1M tokens at launch	Usually predictable, network-bound	Cheap cloud fallback, classification, structured JSON
OpenAI	GPT-4.1	1M	Premium relative to mini tiers	Higher-end but long-context capable	Long-context planning, large document workflows
Azure AI Foundry	Model-agnostic connector to whichever hosted model you choose	Varies by model	Varies by model, region, deployment mode	Varies	Enterprise governance, Microsoft-centric environments, catalog flexibility

The context and cost values above come from public model announcements and technical reports. Llama 3.1 is documented with a 128K window, Gemma 3 with a 131,072-token window, and the Qwen2.5-1M family with up to a 1M context. OpenAI's GPT-4.1 public announcement emphasized a 1M-token window, and Reuters reported GPT-4o mini launch pricing at $0.15 input and $0.60 output per million tokens. Azure AI Foundry availability is intentionally described as model-dependent because Microsoft's catalog changes over time and now spans multiple providers. [5]

The fallback policy should be explicit and enforceable in code. My recommendation is:

1. **Try local first** for private data, light summarization, extraction, classification, and short drafting.
2. **Escalate to low-cost cloud** if the local model times out, fails validation, or produces low-confidence structured output.
3. **Escalate to premium long-context** only for difficult planning, large context sets, or user-approved high-value runs.
4. **Fail closed** when the task is privacy-sensitive and cloud connectors are disallowed.
5. **Log token usage, latency, and fallback reason per step** so you can tune policy over time.

That policy is consistent with the files' emphasis on durable execution, observability, and AI-assisted workflows rather than "just call a model and hope." filecite turn0file1

One security caveat matters here: local LLM infrastructure can still become Internet-facing by accident. Public reporting in early 2026 described large numbers of exposed Ollama instances, despite Ollama being designed for local use. If you expose Ollama or your workflow app beyond localhost, put them behind auth, TLS, and network policy immediately. [6]

Data Handling, Hardware, Tooling, Security, and Licensing

For data handling, the first decision is whether your “local app” is really a **single-user desktop database** or a **shared local server used by a small team**. If it is truly single-user, SQLite can work for some subsystems, but if the product includes queued workers, durable resumptions, execution payloads, workflow version history, secrets, and concurrent editor activity, PostgreSQL is the safer default. The uploaded blueprint’s schema design—workflow entities, workflow history, credentials, executions, execution data, and webhooks—clearly leans toward a relational store with stronger concurrency characteristics. The same material also recommends local data tables and programmatic access patterns for workflow-managed structured data. filecite turn0file1

The recommended local data design is:

Data type	Recommended storage
Users, workflows, versions, credentials metadata, executions, schedules	PostgreSQL
Queue state	Redis
Binary payloads, uploaded files, generated artifacts	Local filesystem first, MinIO optional later
Searchable document chunks and embeddings	PostgreSQL + pgvector, or separate vector store only if scale requires it
Desktop secrets	App-layer encryption plus OS keychain integration where available

That structure matches the uploaded architecture notes around durable state, queue-backed workers, credential storage, and execution history. filecite turn0file1

For privacy and offline-first behavior, design the app so that **all workflow definitions, execution logs, and credentials remain usable without Internet access**, while cloud connectors degrade gracefully rather than blocking the editor. The UI should cache workflow definitions locally; the engine should queue local jobs even if cloud connectors are unavailable; and cloud-bound nodes should clearly display “requires network” status. The uploaded materials strongly support this division between workflow engine durability and external dependency volatility. filecite turn0file1

Encryption should be layered:

- **At rest:** encrypt credentials and API secrets in the application store with a master key; prefer OS full-disk encryption as an outer layer.
- **In transit:** localhost traffic can remain local during development, but anything crossing machine boundaries should use TLS.
- **Secrets handling:** never store provider keys unencrypted in workflow JSON; store references only.

This aligns directly with the uploaded blueprint’s credential guidance, which calls for encrypted secrets at rest and protected handling of the master encryption key. filecite turn0file1

A realistic hardware/OS planning table for local teams is below. These are **engineering estimates**, not vendor-imposed minimums.

Setup profile	Typical use	CPU / RAM	GPU / memory	Storage	OS
Light	Editor + backend + DB, no local LLM	4 cores / 16 GB RAM	None required	50–100 GB SSD	macOS, Linux, Windows
Recommended	Full local stack + Ollama with 7B–12B models	8 cores / 32 GB RAM	Optional 8–12 GB VRAM, or Apple unified memory	100+ GB NVMe	macOS, Linux, Windows
Power user	Multiple workers + bigger local models + heavy testing	12–16 cores / 64 GB RAM	16–24 GB VRAM or high unified memory	200+ GB NVMe	macOS, Linux, Windows

These estimates reflect three things: the uploaded architecture guidance that production-style queue mode typically wants at least midrange CPU/RAM headroom; public Ollama ecosystem guidance that smaller local models can run around the 8–16 GB range while much larger models need far more memory; and the fact that Ollama, Tauri, and Electron all target the major desktop operating systems. filecite turn0file0 filecite turn0file1 [7]

On tooling and workflow, use a **monorepo** from the start. The uploaded blueprint specifically recommends a Turborepo-style structure with shared packages for web, server, worker, engine, sandbox, pieces/connectors, and common schemas. That is the correct shape here because the hardest part of these products is shared contracts: workflow JSON, node definitions, parameter schemas, runtime validation, and execution events. I would add pnpm, shared Zod schemas, Vitest or Jest for unit tests, Playwright for editor/e2e tests, contract tests for connectors, and GitHub Actions or an equivalent CI runner for lint, test, container build, SBOM generation, and signed release artifacts. The monorepo recommendation and shared-type emphasis are strongly supported by the uploaded architecture file. filecite turn0file1

Security is where these products most often fail, and the uploaded blueprint is right to spend so much time on it. The main risks are **sandbox escapes in custom code**, **SSRF through HTTP nodes**, **credential leakage**, **unsafe file/command nodes**, and **unsafe network exposure of local runtimes**. The blueprint highlights sandbox escape classes, dedicated sandbox processes or isolates, memory and timeout limits, SSRF guards, and explicit allow-lists for internal hosts. In addition, public reporting in 2026 showed widespread abuse potential when Ollama or similar local runtimes were accidentally exposed to the public Internet. filecite turn0file1 [6]

The licensing picture is also important if you are “recreating” rather than merely learning from these tools. The uploaded comparison material distinguishes between **closed SaaS** players, **MIT-licensed** platforms such as Activepieces, **AGPL** code-first orchestration like Windmill, and **fair-code / Sustainable Use** style licensing around n8n. That has real consequences. If you plan to fork, redistribute, or commercialize your clone, you must verify the license of any code or SDK you reuse. The same caution applies to Electron, Tauri, Docker Desktop, and model licenses for local LLMs. Tauri uses MIT and Apache 2.0 licensing, Electron is

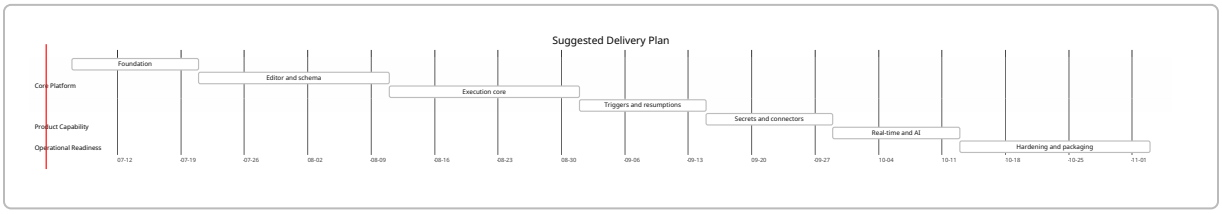
broadly MIT-licensed at the framework layer, and Podman is Apache 2.0; Docker Engine and Docker Desktop have different licensing contexts. filecite turn0file0 filecite turn0file1 [8]

Implementation Plan and Milestones

The implementation plan below assumes a small team of **two to four engineers** with one person able to span backend/infra, one focused on frontend/editor, and one handling connector/runtime/security work. If you have fewer people, extend the schedule rather than compressing the scope.

Phase	Duration	Main deliverables	Exit criteria
Foundation	2 weeks	Monorepo, Compose stack, Postgres, Redis, app shell, auth bootstrap, migrations	Local stack boots reliably on all target OSes
Editor and schema	3 weeks	Workflow JSON schema, node/edge model, React Flow canvas, inspector forms, import/export	Users can create, save, and reload flows
Execution core	3 weeks	Manual runs, worker process, Redis/BullMQ queue, execution state, logs	Users can run simple workflows and inspect results
Triggers and resumptions	2 weeks	Webhook, cron, delay/waitpoint, retry, fail states	Non-trivial workflows can start automatically and resume
Secrets and connectors	2 weeks	Encrypted credentials, HTTP node, file node, data table/db node, first external connector	Real workflows call authenticated APIs safely
Real-time and AI	2 weeks	SSE/WebSocket execution stream, Ollama connector, provider abstraction, structured output validation	Users can watch runs and call local/cloud LLMs
Hardening and packaging	3 weeks	Sandbox isolation, SSRF guard, backups, desktop packaging, test coverage, docs	Team can install and use it locally with confidence

A 17-week plan like this is realistic for a solid internal-quality clone. A thinner prototype can be shown in 6–8 weeks, but it will usually be misleadingly incomplete because it will underinvest in state, security, and debuggability. The attached files make clear that these are not optional details; they are the product. filecite turn0file1



The order matters. Do **not** build lots of connectors before the execution engine, logs, secrets, and resume model are reliable. Doing so creates demo-ware instead of a usable platform. The uploaded blueprints support that prioritization strongly by centering the engine, queue, and durable state ahead of marketplace or polish concerns. filecite turn0file1

Risks, Trade-Offs, Alternatives, and Final Recommendation

The biggest risk is **underestimating the execution engine**. Visual editors are visible and satisfying to build, but the real difficulty lies in deterministic workflow state, retries, resumptions, and failure visibility. The attached architecture file goes into detail on queue lifecycles, waitpoints, suspended states, worker fleets, and durable checkpoints for exactly this reason. If you skimp here, the clone may look right and still be unusable in real work. filecite turn0file1

The second major risk is **unsafe custom code execution**. The uploaded blueprint is right to warn against in-process sandboxing and to emphasize isolated execution modes, timeouts, memory limits, and SSRF protection. In practice, that means your first release should ship with a narrow security posture: disable shell/command execution by default, restrict file access, and prefer a separate sandbox process over trusting the primary app runtime. filecite turn0file1

The third risk is **connector explosion**. Every integration added early imposes schema, auth, testing, rate-limit, and maintenance work. That is why the better product strategy is to ship a strong **HTTP/API node plus a small number of excellent built-ins**, then let your connector SDK mature before broadening the catalog. The comparison file shows that the large ecosystems of Zapier/Make/n8n are differentiated assets; they are not cheap byproducts. filecite turn0file0

The main trade-offs between viable strategies look like this:

Strategy	Speed	Control	Licensing clarity	Long-term maintainability	Recommendation
Greenfield clone	Slowest	Highest	Clean if truly original	Good if architecture is disciplined	Use if you want a platform, not just a tool
Fork Activepieces	Fast	High	Favorable compared with fair-code and closed SaaS options	Good if you accept upstream influence	**Best pragmatic shortcut**
Build on Windmill	Medium	High	Strong copyleft implications	Strong for code-first teams	Better if your users are developers
Automate around n8n	Fast	Medium	Fair-code constraints matter	Good for internal-only customization	Good if you accept n8n constraints

This comparison follows directly from the uploaded files' licensing and platform-positioning analysis. filecite turn0file0 filecite turn0file1

My final recommendation is therefore straightforward:

- If your goal is **the fastest credible local product**, build a **TypeScript modular monolith** with **React + React Flow**, **Fastify/NestJS**, **PostgreSQL**, **Redis/BullMQ**, **filesystem/MinIO**, **SSE**, and **Ollama-first LLM routing**.
- Deploy it first with **Docker Compose** or **Podman**.
- Package it later with **Tauri** if you need a cleaner desktop UX.
- Treat **OpenAI** and **Azure AI Foundry** as optional provider adapters, not architectural anchors.
- If schedule pressure is high, seriously consider **forking/extending Activepieces or using Windmill** rather than cloning the entire category from scratch. filecite turn0file0 filecite turn0file1 [9]

That recommendation is not the most glamorous one, but it is the one most likely to produce a **working, secure, locally operable clone** instead of a convincing but fragile demo.

---

[1] [4] Ollama
https://en.wikipedia.org/wiki/Ollama?utm_source=chatgpt.com

[2] [3] [9] Docker (software)
https://en.wikipedia.org/wiki/Docker_%28software%29?utm_source=chatgpt.com

[5] Llama (language model)
https://en.wikipedia.org/wiki/Llama_%28language_model%29?utm_source=chatgpt.com

[6] Over 175,000 publicly exposed Ollama AI servers discovered worldwide - so fix now
https://www.techradar.com/pro/security/over-175-000-publicly-exposed-ollama-ai-servers-discovered-worldwide-so-fix-now?utm_source=chatgpt.com

[7] Ollama (software)
https://it.wikipedia.org/wiki/Ollama_%28software%29?utm_source=chatgpt.com

[8] Tauri (software framework)
https://en.wikipedia.org/wiki/Tauri_%28software_framework%29?utm_source=chatgpt.com